

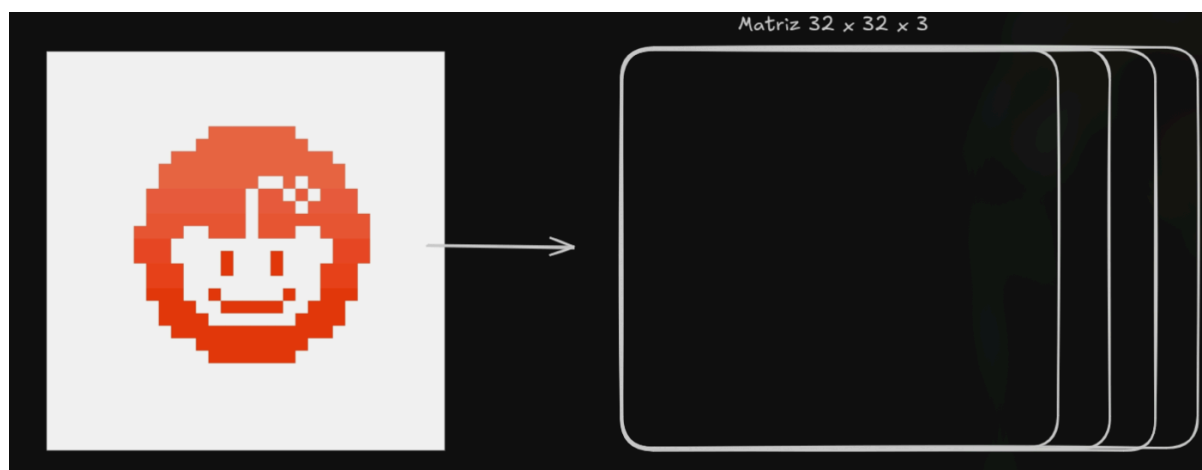
Relatório 16 - Redes Neurais Convolucionais 1 (Deep Learning) (II)

Felipe Fonseca

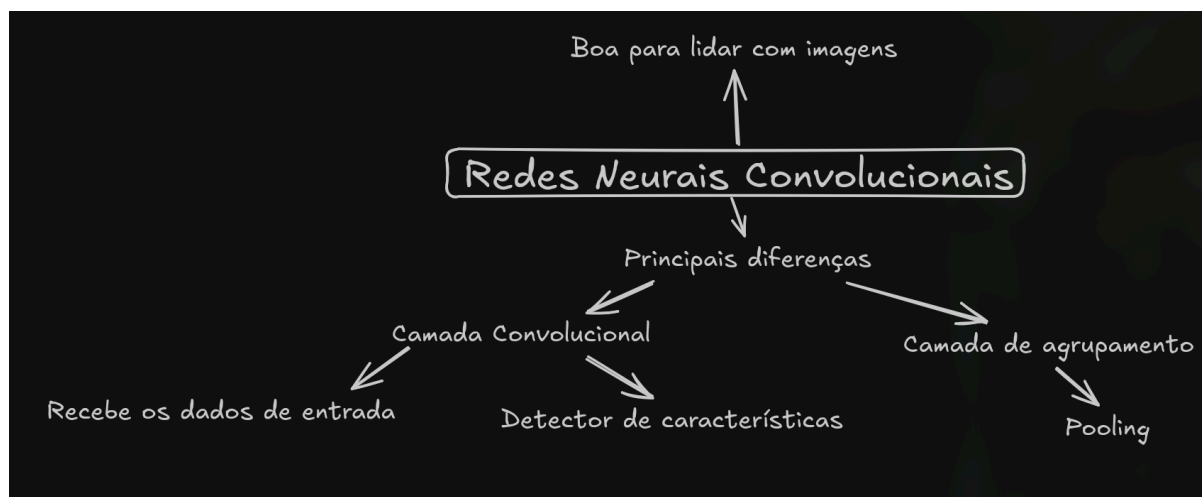
1. Introdução

Nesse card continuamos o estudo das redes neurais em Deep Learning, porém agora estudando as redes neurais convolucionais, aprendendo a lidar com imagens. A atividade começa com a explicação de como é uma imagem para o computador, e posteriormente todo o processo da rede neural até que o modelo entenda a imagem.

2. Desenvolvimento



O primeiro conceito: como é uma imagem em um computador. Uma imagem é uma matriz, contendo pixels na horizontal e na vertical. Cada pixel tem um valor de 0 a 255, onde 0 é o valor mais escuro e 255 o mais claro. Em uma escala de cinza, essas cores são preto e branco. Porém, temos mais cores do que preto e branco. É onde entram as camadas de profundidade, que são normalmente as camadas RGB (Vermelho, Verde e Azul). Atribuindo valores de 0 a 255 para cada pixel, nas 3 camadas, é como obtemos as imagens.



As redes neurais convolucionais podem ser utilizadas para falas e até áudios, mas seu maior uso de destaque é para imagens.

As principais diferenças da rede neural convolucional em termos de funcionamento estão nas camadas convolucionais e nas camadas de agrupamento.

A camada de convolução é a camada que irá lidar com os dados de entrada de uma imagem, e a partir deles, tentar encontrar as principais características, podendo essas características ser as bordas, ou então as principais cores, ou qualquer outro padrão que tenha.

A outra camada é a camada de agrupamento, onde é realizado o Pooling.

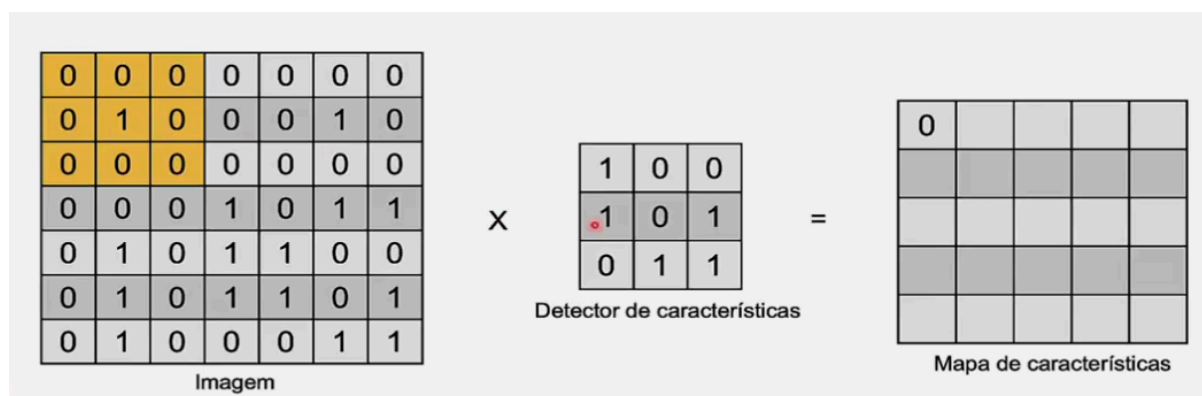


Imagem (1) retirada da videoaula

Essa imagem é um exemplo bem explicativo do que é um detector de características. É uma matriz, normalmente em tamanho 3x3, que para cada valor de referência, multiplica e depois soma todos os resultados, obtendo assim uma imagem menor, mas apenas com as características específicas escolhidas. Esse processo ocorre de forma escondida no código.

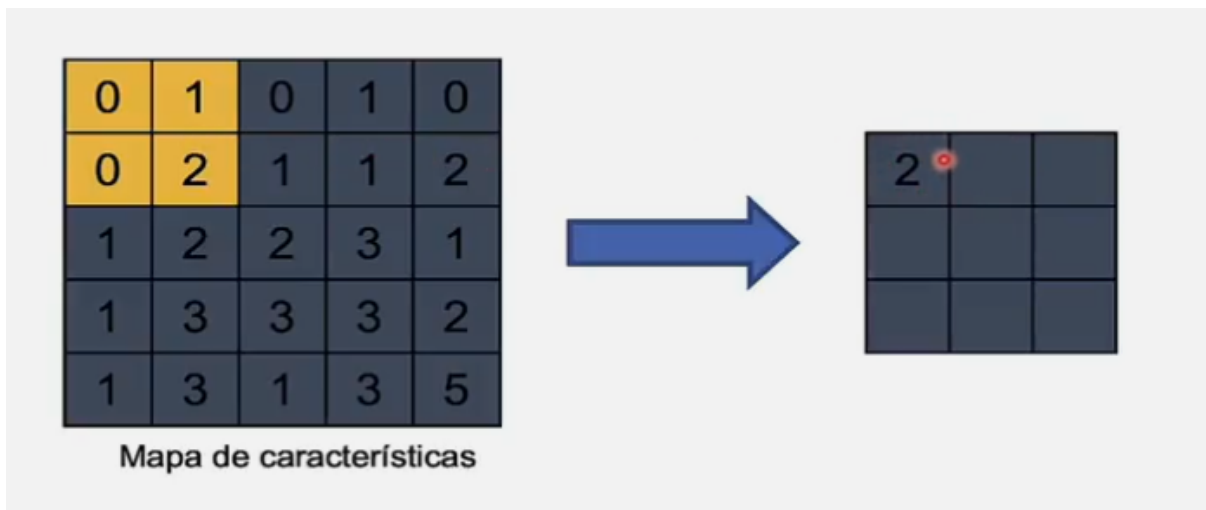


Imagem (2) retirada da videoaula

O Pooling é uma tática de reduzir novamente a matriz, reduzindo a complexidade da imagem e também evitando que o modelo se adapte demais aos dados, causando overfitting. A tática mais comum para o pooling é pegar a matriz do mapa de características, e simplesmente pegar o maior valor. essa técnica é o Max Pooling. Porém em alguns casos também pode ser interessante utilizar outra técnica, que invés do valor máximo, é pego o valor médio, porém não é a melhor opção para o nosso caso.

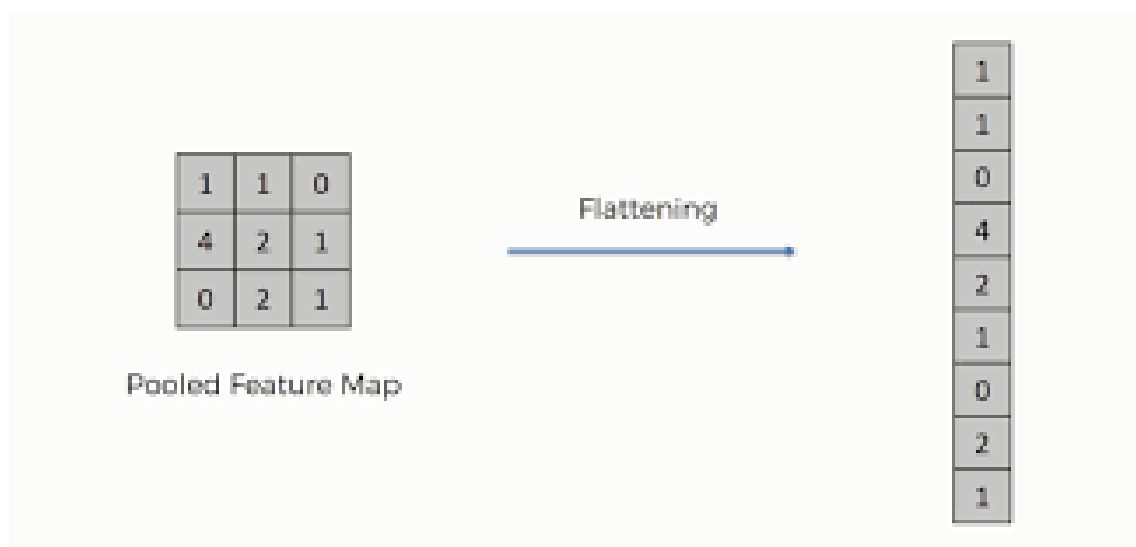
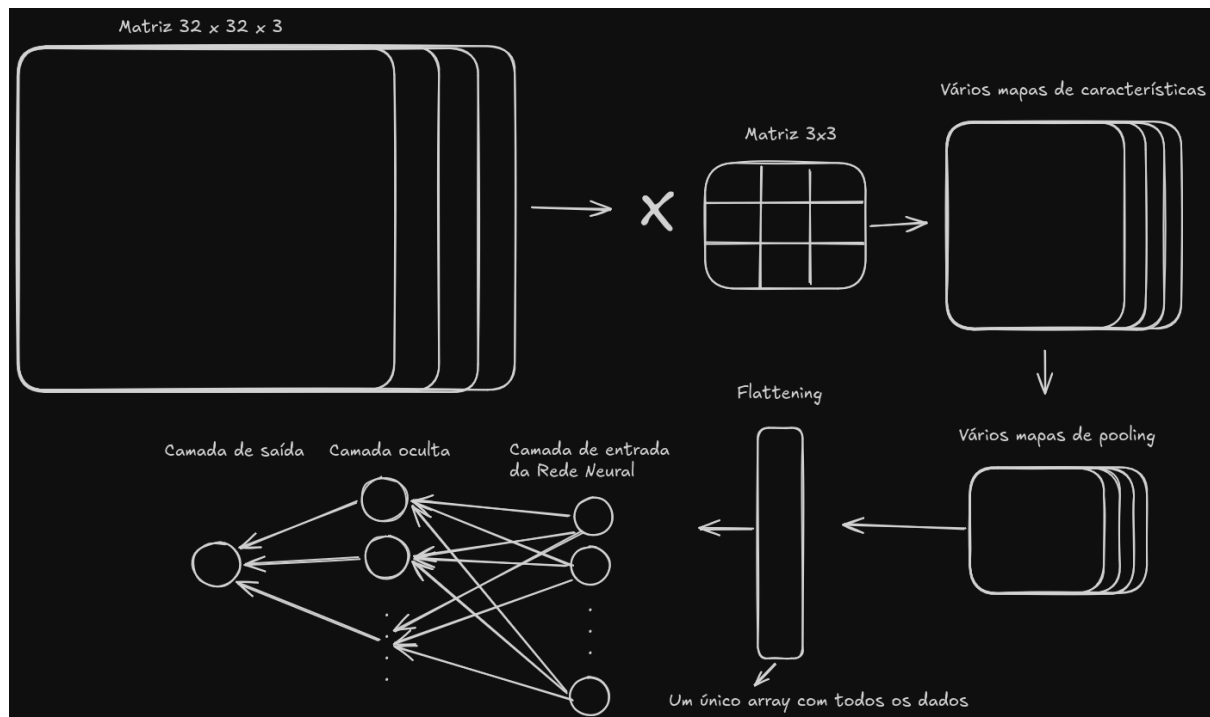


Imagem (3) retirada de super data science

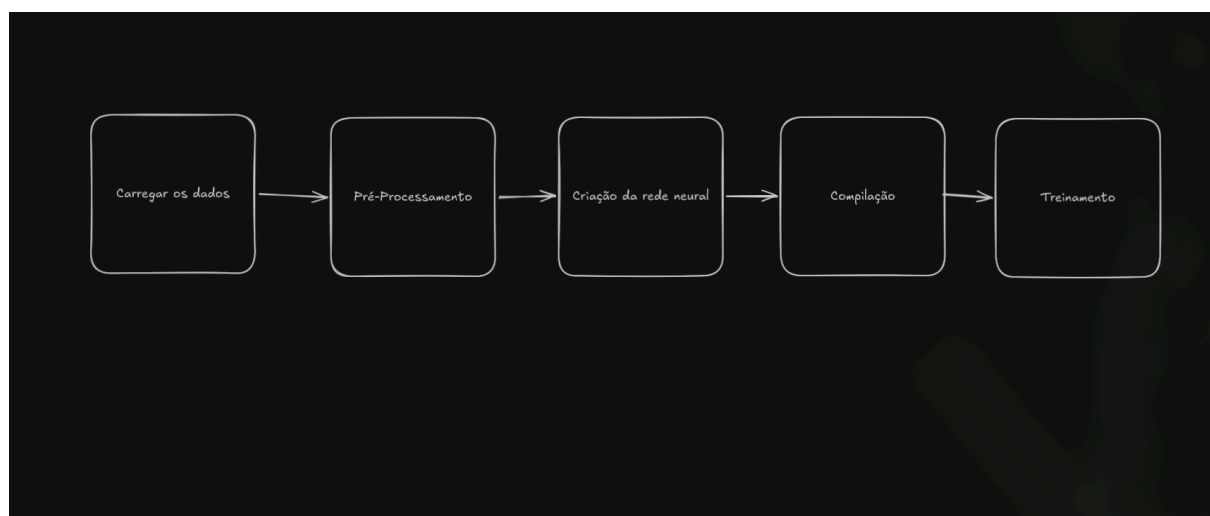
A última etapa é o flattening, que é a transformação do mapa com o pooling para um array. Essa camada é apenas para que os dados possam agora ser enviados para a parte densa da rede neural, que aí sim funciona

exatamente da forma como já foi visto no card passado, com vários neurônios, em camadas escondidas até chegar na camada de resposta.

Exemplo completo de funcionamento de uma Rede Neural Convolucional:



Agora indo para a parte prática do Card. A primeira atividade foi uma rede neural convolucional simples para o dataset mnist, que já vem embutido no Tensor Flow. Esse dataset possui imagens de números escritos à mão, contendo poucos pixels, e com respostas possíveis de 0 à 9, sendo todas as imagens em preto e branco (escala de cinza, apenas uma dimensão de profundidade).



- Carregar os dados: utilizamos o método “load data” para carregar os dados de treino e teste do dataset mnist. Após isso fizemos algumas verificações para identificar a disposição dos dados como tamanho e formato.
- Pré Processamento: após verificar os tamanhos e formatos do dataset, identificamos que o formato dos valores de entrada não estavam considerando a profundidade da imagem, isto é, estava considerando a imagem apenas como a matriz de tamanho 28x28, sem considerar a sua profundidade (1). Para isso, utilizamos o método “reshape”. O segundo passo foi transformar os valores no tipo float para que pudessemos normalizá-los. Considerando que os valores estão entre 255 e 0, ao dividir os dados por 255 conseguimos uma normalização dos dados, tendo todos os valores agora entre 0 e 1. E para finalizar, a resposta apesar de ser um número, não é um valor quantitativo, e sim qualitativo, por isso utilizamos o módulo “utils” da biblioteca Keras para transformar esses números em um array de probabilidade, ou seja, um valor categórico.
- Criação da rede neural: A rede neural foi criada utilizando o método de convolução e depois o pooling, repetindo o processo 2 vezes, e contendo posteriormente 2 camadas densas. O objetivo é conseguir pegar bem as principais características da imagem. Além disso possui camadas de normalização de dados dentro da rede neural e camadas de dropout, tudo isso para evitar o overfitting. Aqui uma representação da rede neural e suas camadas:



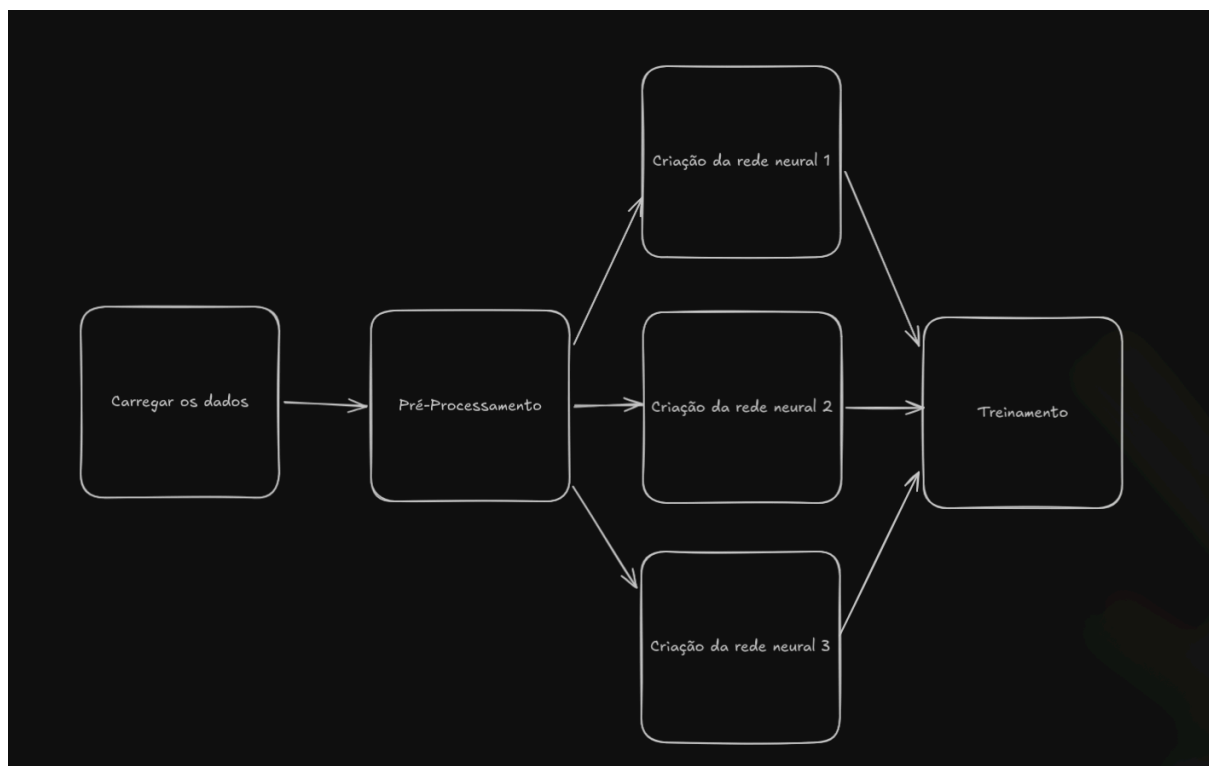
- Compilação: modelo padrão para compilação em modelos categóricos.
- Treinamento: o treinamento foi realizado com batch size de 128 pois havia bastante dados de entrada, apenas 5 épocas pois a ideia era ser rápido, e foi enviado os dados de teste para receber a acurácia e perda real, para poder verificar se o modelo estava aprendendo corretamente ou apenas se adaptando demais aos dados.
- Resultado: O modelo conseguiu uma acurácia de 98% com perda de 3% apenas, um resultado muito bom para esse teste

Para o segundo código, foi repetido todo o processo até a parte do treinamento, onde aqui a diferença foi o treinamento cruzado, ou seja, utilizamos a classe Stratified KFold do sklearn para separar os dados de treino e teste em 5 partes diferentes, e em cada etapa, utilizar 1 das 5 para testar e o resto para treinar. Assim obtendo os resultados e vendo se o algoritmo realmente aprendeu ou então apenas teve sorte com os dados selecionados.

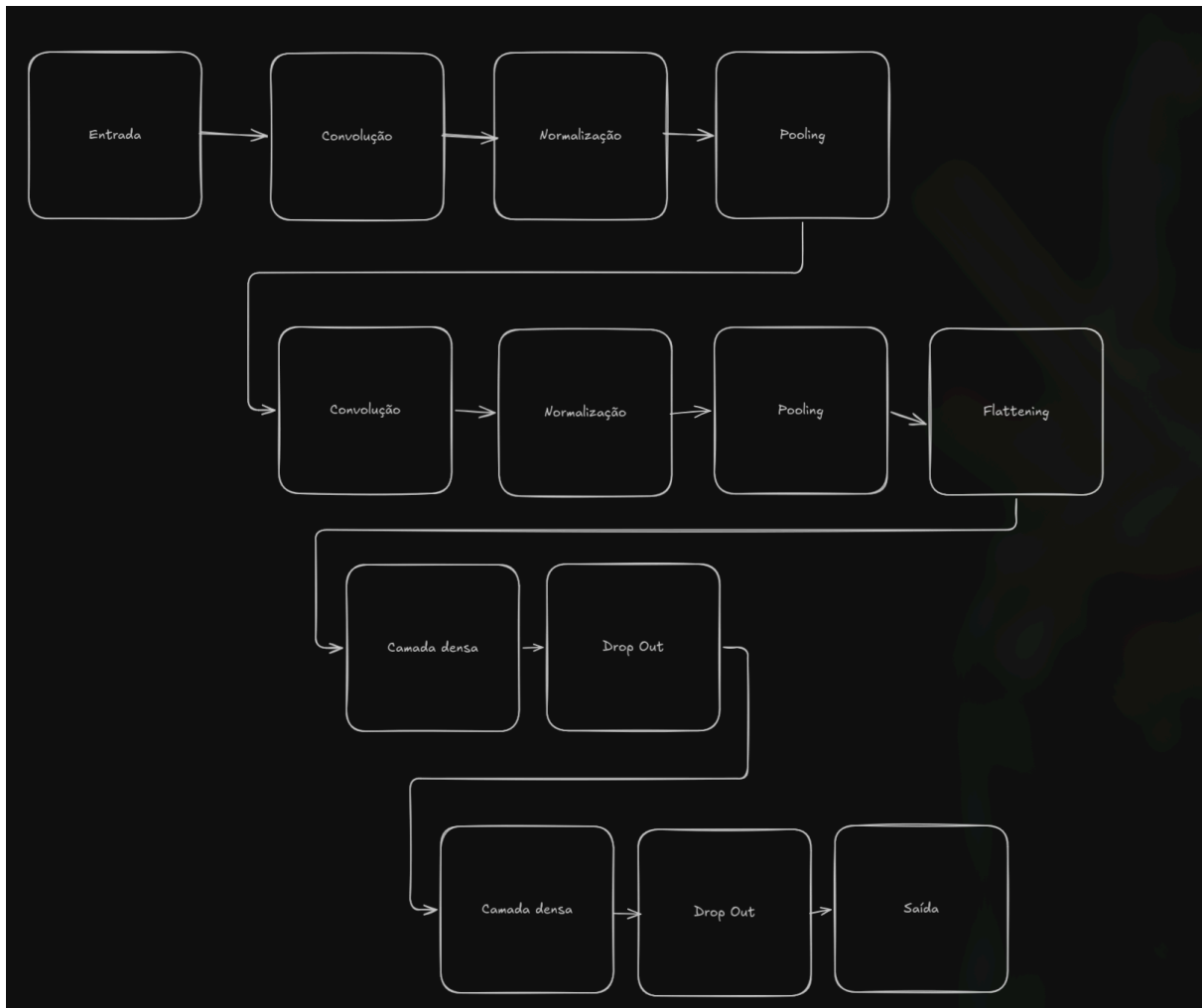
O resultado foi uma acurácia média de 98% e um desvio padrão de 0,2%, ou seja, um resultado muito bom para o treinamento.

Já no terceiro código foi utilizado o Image Data Generator do tensor flow para gerar mais imagens de treino, imagens essas que estariam relacionadas, com zoom e algumas outras condições. Após aplicar o treinamento foi verificado uma acurácia de 97% para esse novo treinamento, o que ainda é um valor muito bom, porém é abaixo do que havíamos conseguido com as imagens padrões. Esse método de criar novas imagens é excelente para ter mais amostras para treinar e ainda em alguns cenários diferentes, onde a imagem não estará em perfeito estado.

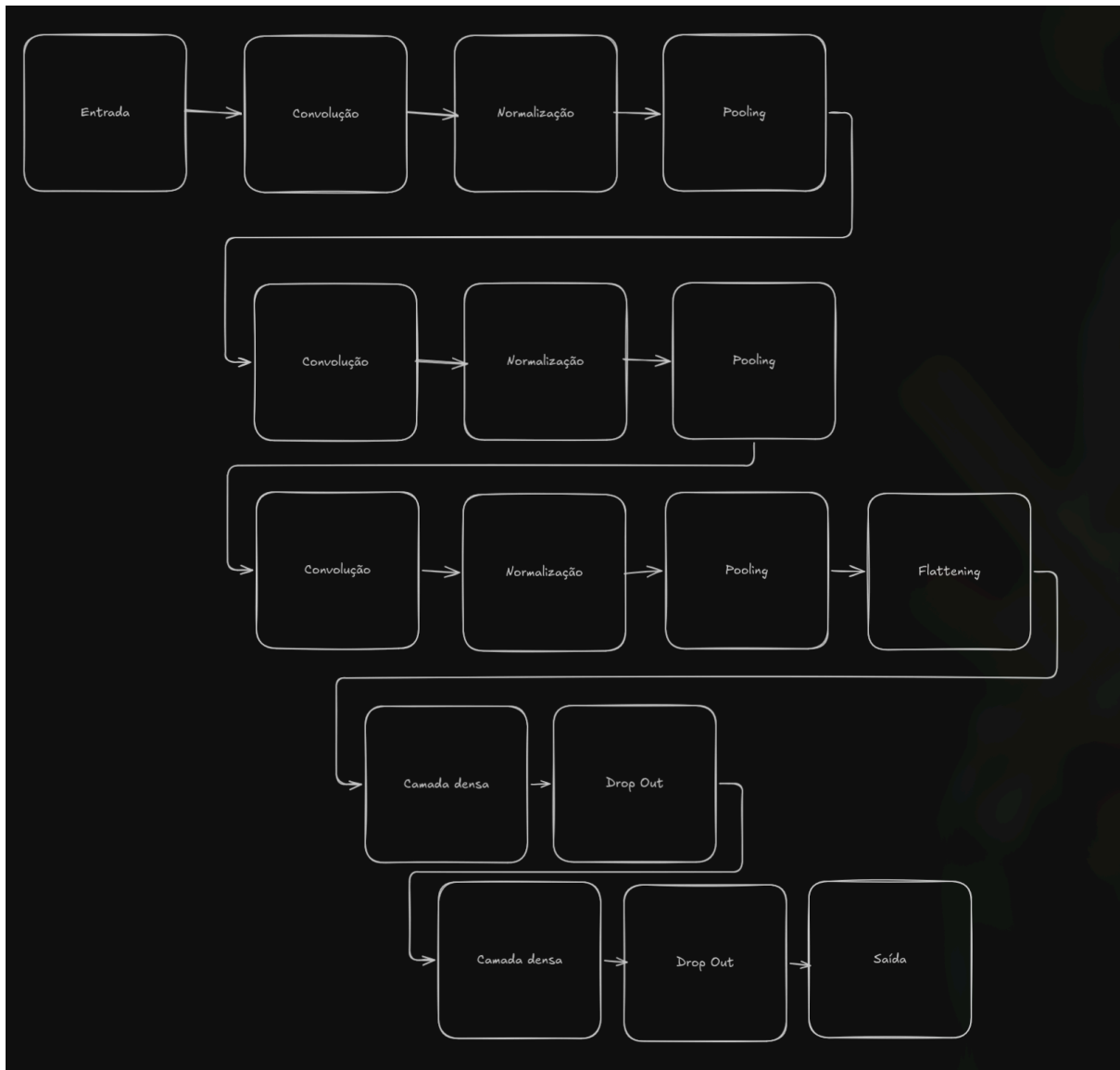
Agora no código prático, eu busquei utilizar uma outra biblioteca que já havia embutida no keras, que é a cifar10. Essa biblioteca possui imagens coloridas, porém ainda em tamanho pequeno, 32x32 pixels. Existem 10 categorias diferentes de imagens, como sapos e caminhões, cada um representado por um número de 0 a 9. O objetivo é o mesmo do código da aula, porém a complexidade aqui é maior visto que a imagem tem muito mais pixels no total, contando com as 3 camadas invés de apenas 1,



- Carregar os dados: inicialmente apenas carreguei os dados e observei sua disposição, tamanhos e formatos assim como anteriormente.
- Pré-Processamento: diferente do dataset anterior, eu não precisei formatar os dados para o formato correto, esse dataset já estava com as 3 camadas corretamente, porém ainda tive que normalizar os números entre 0 e 1 e aplicar a categorização nos resultados para as 10 categorias diferentes.



- Criação da rede neural 1: para a criação da rede neural, o primeiro teste foi apenas uma réplica da rede neural do código feito na aula, apenas com a alteração no tamanho dos dados de entrada, porém a estrutura se manteve exatamente igual.



- Criação da rede neural 2: para a rede neural 2, eu mantive os dois blocos de transformação de imagem com convolução, normalização e pooling, e ainda adicionei mais um extra. Porém a cada bloco eu aumentei a quantidade de filtros (detectores de características), a fim de aprofundar realmente nas características mais importantes das imagens.



- Criação da rede neural 3: Para a terceira rede neural, eu fiz um modelo de convolução 2 vezes seguidas e depois o pooling, assim como havia sido mencionado essa opção em um dos vídeos do treinamento, e foi o que eu fiz. Além disso, adicionei drop outs nesses blocos de transformação também. Os blocos se repetem 3 vezes, e eu mantive a ideia de aumentar os filtros a cada bloco, porém eu também fiz o dropout aumentar a cada bloco, e continuar alto inclusive na parte densa da rede neural. Outra diferença foi aplicar o padding como igual, fazendo com que a matriz não diminuísse de tamanho ao fazer a convolução, apenas ao fazer o pool, isso fez a diferença pois a matriz sumiria e haveria erro por conta da quantidade de camadas de convoluções que foram aplicadas na rede neural.
- Treinamento: para o treinamento, eu mantive os mesmos valores para as 3 redes neurais de teste que eu criei, sendo os valores um batch size de 128 e o treinamento feito em 50 épocas, dessa vez eu não tive escolha e tive que fazer um treinamento demorado para obter um bom resultado.

Resultados

Métrica	Rede Neural 1	Rede Neural 2	Rede Neural 3
Accuracy (Treino)	0.9406	0.9718	0.9030
Loss (Treino)	0.1749	0.0840	0.3036
Val_Accuracy (Validação)	0.7056	0.7450	0.8596
Val_Loss (Validação)	1.4032	1.4841	0.4632

Resultados: através dos resultados apresentados, é possível observar que os dois primeiros modelos conseguiram um percentual de acerto muito alto durante o treinamento, mas ao utilizarmos os dados de teste para avaliarmos, o resultado foi abaixo, o que indica que provavelmente o modelo estava se adaptando demais aos dados, causando o overfitting. Enquanto isso, a rede neural 3 possui uma porcentagem de acertos abaixo no treinamento, cerca de 6% a menos do que os outros, porém no teste real ele teve mais de 10% a mais, mostrando que é o melhor modelo e que aprendeu mais os dados do que os outros, que aparentemente apenas decoraram os padrões.

3. Conclusão

As redes neurais convolucionais funcionam muito bem para o treinamento de imagens dado a sua capacidade de pegar as principais características de uma imagem e mantê-las mesmo reduzindo o tamanho de suas matrizes drasticamente. Gostei muito do aprendizado dessa aula, e principalmente da prática, onde utilizei de um dataset diferente e pude fazer

meus próprios testes a fim de encontrar o melhor design para a rede neural funcionar e obter um resultado gratificante.

4. Referências

Videoaulas do card;

Documentação do Keras <https://keras.io/api/datasets/cifar10/>;

Blog para dúvida:

<https://www.ibm.com/br-pt/think/topics/convolutional-neural-networks>.

Imagem 3:

<https://www.superdatascience.com/blogs/convolutional-neural-networks-cnn-step-3-flattening>.